A04 — Definir estratégia de persistência

1. Objetivo

Definir a estratégia de persistência do SmartLog utilizando Cloud Firestore e estabelecer como o Spring Boot realizará leitura e escrita dos dados, incluindo:

- organização das coleções;
- modelagem documental;
- mapeamento entre documentos Firestore e objetos Java;
- isolamento multi-tenant;
- relacionamento entre Entregas, Rotas, Clientes e Motoristas;
- compatibilidade com autorização baseada em perfil.

O A02 define o Cloud Firestore como camada de persistência responsável pelos registros operacionais de entregas, frotas, motoristas e empresas.

---

2. Estratégia escolhida

O SmartLog utilizará **Cloud Firestore como banco NoSQL documental**, acessado exclusivamente pelo backend Spring Boot.

O fluxo de persistência será:

```
Flutter / Angular
       │
       │ HTTPS + Firebase ID Token
       ▼
Spring Boot REST API
       │
       │ valida JWT
       │ extrai uid / role / empresaId
       ▼
Application / Service
       │
       ▼
Firestore Repository
       │
       ▼
Cloud Firestore
```

Essa abordagem mantém a regra de negócio e autorização no backend.

O A02 estabelece que o Spring Boot valida o token, aplica RBAC e realiza as leituras/escritas no Firestore de forma isolada por `empresaId`.

---

3. Justificativa do Cloud Firestore

A manutenção do Firestore é coerente com a arquitetura porque o projeto necessita de:

3.1 Baixa latência

O Firestore é utilizado para armazenamento dos registros operacionais e oferece uma estrutura adequada para consultas documentais.

3.2 Atualizações em tempo real

A arquitetura considera alterações de status das entregas refletidas rapidamente nas interfaces, reduzindo a necessidade de polling contínuo.

3.3 Isolamento multi-tenant

A organização dos dados permite separar logicamente os dados por `empresaId`.

3.4 Escalabilidade

O Firestore oferece uma abordagem serverless compatível com a arquitetura proposta e integração com o ecossistema Firebase/Google Cloud.

---

4. Estrutura das coleções

A estratégia adotada será baseada em uma coleção por empresa, utilizando subcoleções para os recursos operacionais.

```
empresas/
└── {empresaId}/
    ├── clientes/
    │   └── {clienteId}
    │
    ├── motoristas/
    │   └── {motoristaId}
    │
    ├── rotas/
    │   └── {rotaId}
    │
```

```
    └── entregas/
        └── {entregaId}
```

Essa estrutura faz com que o `empresaId` faça parte do caminho físico do documento.

Exemplo:

```
empresas/empresa-001/entregas/entrega-001
```

em vez de depender exclusivamente de:

```
entregas/entrega-001
```

com um campo `empresaId`.

A primeira abordagem reduz o risco de uma operação do backend consultar acidentalmente documentos de outra organização.

---

5. Documento Empresa

```
empresas/{empresaId}
```

Exemplo:

```
{
  "nome": "Empresa ABC",
  "ativo": true,
  "criadoEm": "2026-08-19T10:00:00Z",
  "atualizadoEm": "2026-08-19T10:00:00Z"
}
```

O documento representa o tenant corporativo.

As entidades operacionais pertencem a uma empresa por meio da hierarquia da coleção.

---

6. Documento Cliente

```
empresas/{empresaId}/clientes/{clienteId}
```

Modelo:

```
{
  "id": "cliente-001",
  "nome": "Cliente Exemplo",
  "email": "cliente@example.com",
  "telefone": "11999999999",
  "ativo": true,
  "criadoEm": "2026-08-19T10:00:00Z",
  "atualizadoEm": "2026-08-19T10:00:00Z"
}
```

O documento não precisa repetir `empresaId`, pois a empresa já está representada no caminho.

Entretanto, caso o domínio ou consultas futuras exijam explicitamente o campo, ele poderá ser mantido como informação redundante controlada pelo backend.

---

7. Documento Motorista

```
empresas/{empresaId}/motoristas/{motoristaId}
```

Modelo:

```
{
  "id": "motorista-001",
  "uid": "firebase-user-id",
  "nome": "João da Silva",
  "email": "joao@example.com",
  "ativo": true,
  "criadoEm": "2026-08-19T10:00:00Z",
  "atualizadoEm": "2026-08-19T10:00:00Z"
}
```

O `uid` representa a associação entre o motorista e sua identidade no Firebase Authentication.

O A05 determina que usuários são autenticados pelo Firebase e que o sistema trabalha com identificação do usuário e perfil.

No cadastro realizado pelo ADMIN, o backend deve associar automaticamente o motorista à empresa autenticada.

---

8. Documento Rota

```
empresas/{empresaId}/rotas/{rotaId}
```

Modelo:

```
{
  "id": "rota-001",
  "nome": "Rota Zona ABC",
  "motoristaId": "motorista-001",
  "status": "PLANEJADA",
  "criadaEm": "2026-08-19T10:00:00Z",
  "atualizadaEm": "2026-08-19T10:00:00Z"
}
```

As referências para entidades relacionadas serão armazenadas por ID.

A rota não deve duplicar os dados completos do motorista.

---

9. Documento Entrega

```
empresas/{empresaId}/entregas/{entregaId}
```

Modelo:

```
{
  "id": "entrega-001",
  "codigo": "ENT-001",
  "clienteId": "cliente-001",
  "rotaId": "rota-001",
  "motoristaId": "motorista-001",
  "status": "PENDENTE",
  "origem": "São Paulo - SP",
```

```
  "destino": "Santo André - SP",
  "criadaEm": "2026-08-19T10:00:00Z",
  "atualizadaEm": "2026-08-19T10:00:00Z"
}
```

As relações são representadas por IDs:

```
clienteId
rotaId
motoristaId
```

Não será realizada duplicação dos documentos relacionados dentro da entrega.

---

10. Mapeamento Java / Firestore

A aplicação Spring Boot deverá separar o domínio da infraestrutura de persistência.

Estrutura recomendada:

```
domain/
├── model/
│   ├── Entrega
│   ├── Rota
│   ├── Cliente
│   └── Motorista
│
application/
├── service/
│
infrastructure/
└── persistence/
    └── firestore/
        ├── FirestoreEntregaRepository
        ├── FirestoreRotaRepository
        ├── FirestoreClienteRepository
        └── FirestoreMotoristaRepository
```

O domínio não deve depender diretamente das classes do Firebase.

A infraestrutura será responsável por converter:

```
Firestore Document
        ↓
Persistence Model
        ↓
Domain Model
```

e no sentido inverso:

```
Domain Model
        ↓
Persistence Model
        ↓
Firestore Document
```

Isso evita acoplamento do domínio à tecnologia de banco escolhida.

---

11. Acesso ao Firestore

O Spring Boot utilizará o SDK/cliente oficial do Firebase Admin para realizar as operações no Firestore.

O acesso deve ficar restrito à camada de infraestrutura.

Exemplo conceitual:

```
EntregaService
      │
      ▼
EntregaRepository
      │
      ▼
FirestoreEntregaRepository
      │
      ▼
Firebase Admin SDK
      │
      ▼
Cloud Firestore
```

Controllers não devem executar operações diretamente no Firestore.

---

12. Isolamento Multi-Tenant

O isolamento é uma das principais regras da persistência.

O `empresaId` será determinado pelo contexto autenticado.

```
Firebase Token
      │
      ▼
Security Filter
      │
      ├── uid
      ├── role
      └── empresaId
            │
            ▼
       Application Layer
            │
            ▼
       Repository
            │
            ▼
empresas/{empresaId}/...
```

O cliente não poderá informar livremente:

```
{
  "empresaId": "outra-empresa"
}
```

para acessar dados externos.

O backend deve utilizar exclusivamente o `empresaId` obtido do usuário autenticado.

Essa regra está diretamente alinhada ao A02, que determina que todas as consultas e gravações sejam filtradas pelo `empresaId` do usuário logado.

---

13. Autorização por perfil

O isolamento por empresa não substitui o RBAC.

As duas regras devem ser aplicadas:

```
Autorização =
    Perfil permitido
    +
    empresaId correspondente
    +
    escopo do recurso
```

ADMIN

Pode acessar os recursos da própria empresa.

```
empresaId = empresa autenticada
role = ADMIN
```

MOTORISTA

Além do `empresaId`, o backend deve verificar se o recurso está associado ao motorista autenticado.

Exemplo:

```
empresaId = empresa autenticada
role = MOTORISTA
motoristaId = motorista autenticado
```

O A05 define que o motorista somente pode acessar entregas e rotas vinculadas ao seu perfil e empresa.

CLIENTE

Além do `empresaId`, o backend deve restringir o acesso às próprias entregas.

```
empresaId = empresa autenticada
role = CLIENTE
clienteId = cliente autenticado
```

O A05 determina que o cliente tenha acesso exclusivamente ao histórico de seus próprios pedidos.

---

14. Consultas por escopo

Exemplo de consulta administrativa:

```
empresas/{empresaId}/entregas
```

Exemplo de consulta do motorista:

```
empresas/{empresaId}/entregas
WHERE motoristaId == motoristaAutenticado
```

Exemplo de consulta do cliente:

```
empresas/{empresaId}/entregas
WHERE clienteId == clienteAutenticado
```

Dessa forma, a camada de persistência recebe o contexto de autorização necessário para construir a consulta correta.

---

15. Relacionamentos

O Firestore não utilizará joins relacionais tradicionais.

Os relacionamentos serão representados por identificadores.

```
Entrega
 ├── clienteId ──────► Cliente
 ├── rotaId ─────────► Rota
 └── motoristaId ────► Motorista
```

Isso mantém os documentos pequenos e evita duplicação desnecessária.

Quando a API precisar apresentar informações agregadas, a camada de aplicação poderá buscar os documentos relacionados e montar o DTO de resposta.

---

16. Consistência e atualização de status

As alterações críticas de status deverão ser realizadas pelo backend.

Exemplo:

```
PENDENTE
    ↓
EM_ROTA
    ↓
ENTREGUE
```

O cliente não deve possuir permissão para alterar esse estado.

O motorista poderá realizar a atualização dentro de seu escopo autorizado.

O ADMIN poderá realizar a alteração operacional conforme sua permissão.

Essa regra é consequência direta do modelo RBAC definido no A05.

---

17. Datas

Os documentos deverão utilizar timestamps compatíveis com o Firestore.

Campos recomendados:

```
criadaEm
atualizadaEm
```

As datas devem ser geradas/controladas pelo backend sempre que representarem eventos internos do sistema.

O frontend não deve possuir autoridade para definir arbitrariamente campos de auditoria.

---

18. Índices

Os índices necessários deverão ser criados conforme as consultas utilizadas pela API.

Consultas previstas incluem:

```
entregas por status
entregas por rota
entregas por motorista
entregas por cliente
rotas por motorista
```

A necessidade de índices compostos deve ser avaliada conforme as consultas efetivamente implementadas.

Não devem ser criados índices especulativos sem uma consulta que os utilize.

---

19. Responsabilidades por camada

Controller

Responsável por:

- receber HTTP;
- validar request;
- extrair parâmetros;
- delegar para a aplicação;
- transformar resultado em response.

Não deve acessar Firestore diretamente.

Application / Service

Responsável por:

- executar casos de uso;
- aplicar regras de negócio;
- validar operações;
- coordenar repositories;
- respeitar o contexto de autorização.

Repository

Responsável por abstrair a persistência:

```
EntregaRepository
RotaRepository
```

```
ClienteRepository
MotoristaRepository
```

Firestore Repository

Responsável por:

- montar paths de documentos;
- executar consultas;
- converter documentos;
- persistir alterações.

Firebase Authentication

Responsável pela identidade e autenticação dos usuários.

O Firebase fornece a camada primária de identidade do sistema.

---

20. Segurança das credenciais

As credenciais utilizadas pelo Firebase Admin SDK não serão armazenadas no código-fonte.

A arquitetura define que Service Accounts e chaves das integrações externas permaneçam restritas ao backend por meio de variáveis de ambiente.

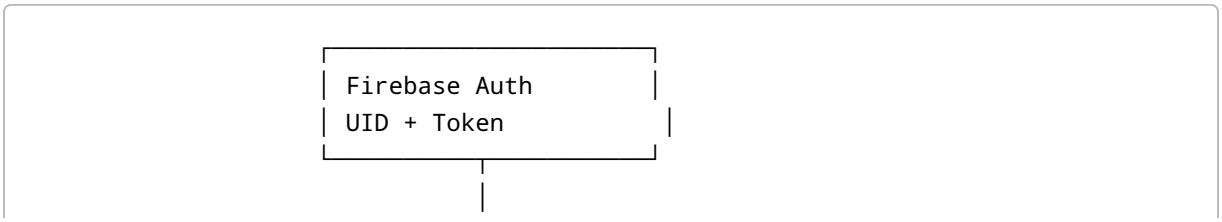
O Flutter e o Angular jamais deverão receber:

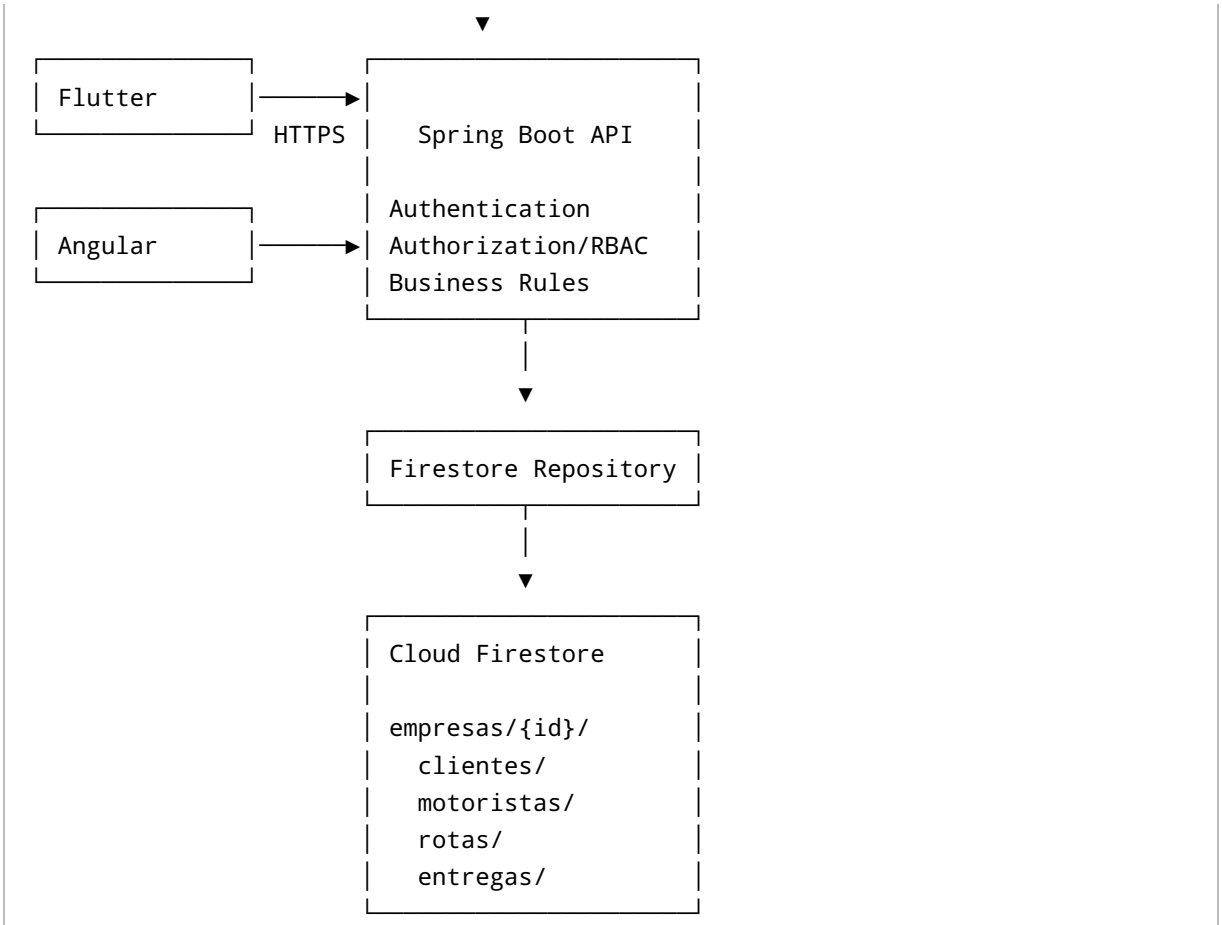
- Service Account;
- credenciais administrativas do Firebase;
- chaves privadas;
- credenciais de OpenAI/Gemini.

---

21. Estratégia de persistência final

A estratégia final pode ser resumida como:





Critérios de aceite

- [x] Estratégia de utilização do Firestore definida.
- [x] Acesso ao Firestore centralizado no Spring Boot.
- [x] Estrutura de coleções definida.
- [x] Mapeamento de Entregas definido.
- [x] Mapeamento de Rotas definido.
- [x] Mapeamento de Clientes definido.
- [x] Mapeamento de Motoristas definido.
- [x] Isolamento multi-tenant por `empresaId` definido.
- [x] Estratégia de autorização por ADMIN, MOTORISTA e CLIENTE definida.
- [x] Separação entre domínio e infraestrutura definida.
- [x] Estratégia de credenciais definida.
- [x] Consultas prioritárias identificadas.
- [x] Estratégia compatível com Flutter e Angular.